

Reporte de Agentic Loop

1. ANÁLISIS DE ARQUITECTURA

ANÁLISIS ARQUITECTÓNICO - Validador de Prompts y Tokens

PATRÓN ARQUITECTÓNICO: Single-File Dual-Layer Architecture

Dada la restricción crítica de máximo 2 archivos, se adopta una arquitectura ultra-condensada donde cada archivo representa una capa completa del sistema. Esta aproximación es válida para herramientas internas de baja fricción y prototipado rápido de utilidades DevSecOps.

CAPAS DEL SISTEMA:

1. Capa de Presentación (index.html): Monolito frontend autocontenido con HTML semántico, estilos Tailwind via CDN y lógica JS vanilla. Gestiona el ciclo completo UI: validación de entrada, estados de carga, renderizado de resultados y manejo de errores de red.
2. Capa de Servicio (main.py): Microservicio FastAPI con un único endpoint POST /api/analyze. Implementa lógica de estimación de tokens ($\text{words} * 1.3$), simulación de latencia asíncrona no bloqueante con `asyncio.sleep()` y clasificación de estado (Óptimo / Alerta).

DECISIONES DE SEGURIDAD (DevSecOps):

- CORS configurado explícitamente para orígenes localhost únicamente, bloqueando cualquier origen externo no autorizado.
- Validación de payload en backend con Pydantic BaseModel para prevenir inyección de datos malformados.
- Validación defensiva en frontend antes de cualquier llamada de red (fail-fast client-side).
- Sin persistencia de datos, sin autenticación requerida para herramienta interna local, superficie de ataque mínima.

TECNOLOGÍAS SELECCIONADAS:

- Backend: Python 3.10+, FastAPI, Uvicorn, asyncio (stdlib), Pydantic v2.
- Frontend: HTML5, Tailwind CSS v3 via CDN, JavaScript ES2020+ (fetch API, async/await, template literals).
- Sin base de datos, sin dependencias externas adicionales, sin frameworks JS pesados.

FLUJO DE DATOS:

Usuario escribe prompt ! Validación JS (vacío = borde rojo, abort) ! fetch POST /api/analyze ! FastAPI valida Pydantic ! `asyncio.sleep(random 0.5-2s)` ! calcula tokens y estado ! JSON response ! JS renderiza 3 tarjetas de resultado.

ESTADOS DE LA UI:

1. Idle: textarea habilitado, botón activo.
2. Error local: textarea con borde rojo, sin petición de red.
3. Loading: botón deshabilitado con texto 'Analizando...', spinner visual.
4. Success: panel de resultados visible con 3 tarjetas (Tokens, Latencia, Estado).
5. Error de red: mensaje de error amigable sin romper la UI.

CONSIDERACIONES DE RENDIMIENTO:

La simulación asíncrona con `asyncio.sleep()` garantiza que el event loop de FastAPI no se bloquee durante la latencia simulada, permitiendo concurrencia real si múltiples usuarios usan la herramienta simultáneamente en entorno de equipo.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.py | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 15,561

Tokens de salida (output): 7,976

Total tokens: 23,537

Horas-hombre estimadas: ~2.8 horas

Modelo utilizado: claude-sonnet-4-6